Stela System Architecture: A Comprehensive Refactoring Framework for Agentic Integrity and Semantic Truth

1. Introduction: The Imperative for a Repository-Resident Operating System

The contemporary landscape of software engineering is undergoing a profound transformation, driven largely by the integration of Large Language Models (LLMs) into the development lifecycle. However, the prevailing integration patterns often suffer from significant "intelligence entropy" and "context rot." The legacy nukeCE repository, while robust in its foundational principles of "Project Truth" and "State of Play," faces scalability challenges inherent to document-based governance. This report outlines a rigorous architectural refactoring plan to transition the repository into **Stela**—a repository-resident operating system designed to enforce agentic integrity through mechanical contracts and epistemic honesty.

The transition to Stela is not merely a renaming exercise but a fundamental realignment of the repository's physics. It moves from a model of "suggested constraints," where governance relies on the vigilance of human operators, to "machine-enforced contracts," where the repository itself acts as an active Context Engine. This shift addresses the core vectors of system degradation identified in the Anti-Drift Doctrine: configuration drift, where infrastructure diverges from intent; architectural drift, where logic evolves without map updates; and semantic drift, where the "hallucination of coherence" masks underlying decay.

By centralizing operational logic into a cohesive /tools directory, establishing a single authoritative TRUTH.md, and formalizing the "Stela" identity within the initialization sequence, we establish a "Pointer-Only Architecture". This ensures that the system survives the stochastic nature of AI interaction, replacing the probabilistic guessing of "helpful" assistants with the deterministic proof required by high-stakes engineering. The following sections detail the specific refactoring mandates required to elevate Stela from a passive archive to a dynamic, self-policing operating environment.

2. The Truth Layer: Refactoring State Management

The most critical component of the Stela architecture is its handling of "Truth." In the legacy system, truth was bifurcated between PROJECT_TRUTH.md (the constitution) and docs/library/MEMENTOS.md (episodic history), with STATE_OF_PLAY.md acting as a status log. This fragmentation creates a "split-brain" risk where an agent may prioritize obsolete episodic data over current architectural invariants. The refactoring consolidates these into a rigorous semantic hierarchy.

2.1 Consolidating PROJECT_TRUTH.md into TRUTH.md

The file currently known as PROJECT_TRUTH.md defines the project's immutable core values,

such as the "Forever Fork" philosophy, the "Security First" mandate, and the "PHP 8.2+" architectural north star. However, its coexistence with MEMENTOS.md introduces noise. Mementos, by definition, are point-in-time reflections—episodic memories that decay rapidly. A production-grade context engine requires semantic memory: distilled, timeless facts that remain true regardless of temporal distance.

2.1.1 The Scrub Protocol and Invariant Extraction

The transformation requires executing a "Scrub Protocol" to migrate valid business logic from MEMENTOS.md into the new TRUTH.md. This is a forensic audit process. We must parse the historical logs for "Invariants"—decisions that have hardened into architectural laws. For instance, MEM-002 documents a deep-dive security audit that established a zero-tolerance policy for legacy SQL injection. In the Stela architecture, the *event* of the audit is historical trivia, but the *outcome* (Strict SQL compliance) is an invariant.
Therefore, the new TRUTH.md will absorb these invariants. The entry "MEM-003: Migration to PHP 8.2 Baseline" ceases to be a memento of a past event and becomes a compilation rule in TRUTH.md: "Target Runtime: PHP 8.2+ (Strict Types)." Once these semantic kernels are extracted, the MEMENTOS.md file is deprecated and deleted ("composted") to reduce the cognitive load on the agent. This adheres to the "Token Economics" principle, where every file in the context window must "pay rent" by providing a high signal-to-noise ratio.

2.1.2 The Structure of the New TRUTH.md

The new TRUTH.md serves as the Single Source of Truth (SSoT). To function effectively within the "Open Prompt," it must follow a strict schema that allows the open tool to parse and inject it deterministically.

- **Identity & Mission:** Defines the repository as php-nuke-ce (Stela), a "Forever Fork" maintained for archival integrity and modern security.
- **Architectural Invariants:** Hard constraints extracted from the Scrub Protocol, such as "No logic in public_html root" or "All modules must be namespaced."
- **Operational Laws:** Governance rules that are non-negotiable, specifically the prohibition of direct pushes to main and the requirement for PR-only workflows.
- **Decision Log (ADRs):** A concise index of Architectural Decision Records that supersede prior logic, ensuring the agent acts on "Case Law" rather than obsolete precedent.

2.2 Refactoring STATE_OF_PLAY.md to SoP.md

The renaming of STATE_OF_PLAY.md to SoP.md is more than a change in nomenclature; it signals a shift in function from a passive "log" to an active "Context Container". In the legacy nukeCE system, the state file was often treated as a diary. In Stela, SoP.md is the dynamic map of the territory—the "Save Game" state that allows a stateless agent to rehydrate instantly.

2.2.1 The Context Container Concept

An agent operating in the Stela architecture is stateless. It has no memory of previous sessions. Therefore, SoP.md must contain 100% of the context required to perform the next action. If a piece of information is necessary for decision-making but is missing from SoP.md, the system is in a failure state known as "Drift".

The refactored SoP.md must serve as the primary anchor for the agent's short-term memory. It acts as the bridge between the immutable laws of TRUTH.md and the immediate objectives of TASK.md. This requires the file to be updated after *every* successful commit. A code change without a corresponding SoP.md update is considered a violation of the system's physics, effectively "ghost code" that exists in the repository but not in the agent's reality.

2.2.2 Mandatory Schema Fields

To enforce this utility, SoP.md requires a rigid schema:

- **Current Phase:** A macro-level descriptor of the project's momentum (e.g., "Phase 2: Toolchain Migration"). This prevents the agent from suggesting features relevant to Phase 1 or Phase 3.
- **Active Blockers:** A specific list of impediments. This allows the agent to prioritize unblocking tasks over new feature development, adhering to the "Weighted Shortest Job First" (WSJF) logic.
- **Architecture Map:** An ASCII representation of the system topology. As identified in the "Anti-Drift Doctrine," architectural drift occurs when the code evolves but the map does not. Including a live map in SoP.md forces the operator to acknowledge structural changes.
- **Recent Context:** A rolling summary of the last 3-5 completed tasks. This replaces the "chat history" of a conversational interface, providing a high-fidelity, curated short-term memory that is free of the hallucinations often found in raw chat logs.

2.3 The Task Singularity: Converting Dispatch Packets to TASK.md

The legacy "Dispatch Packet" (ops/templates/DISPATCH_PACKET_TEMPLATE.md) functioned as a work order—a transient instruction set passed to a worker. While effective for delegation, it lacked persistence. The Stela architecture replaces this with TASK.md, a file that represents a "Singularity of Focus."

2.3.1 From Queue to State

TASK.md is not just a set of instructions; it is a persistent state object that exists at the root of the repository until the work is complete. It represents a contract between the Operator and Stela. By reifying the task as a file, we enable the agent to "check" its objective against the repo state at any time, preventing the "Context Drift" that occurs during long, multi-step reasoning chains.

2.3.2 The Schema of Integrity

The TASK.md file must adhere to a schema that enforces the "Anti-Drift" principles:

- **Objective:** A single, unambiguous sentence defining the goal. Ambiguity here is treated as damage.
- **Context Links:** Explicit pointers to the specific sections of SoP.md or TRUTH.md that constrain the task. This enforces the "Pointer-Only Architecture," preventing the task from containing stale copies of rules.
- **Constraints:** Binary limitations (e.g., "Do not modify core.php"). These are permissions, not preferences, and are mechanically enforced.

- **Definition of Done (Receipts):** This is the most critical innovation. The task is not done when the agent *says* it is done; it is done when verifiable "Receipts" are generated. These receipts—Git commit hashes, test runner outputs, ASCII dumps of visual changes—serve as the "Currency of Trust".

Feature	Legacy Dispatch Packet	Stela TASK.md	Theoretical Implication
Persistence	Transient (Chat-based)	Persistent (File-based)	Externalizes "Intent" from the agent's RAM to the Repo's HDD.
Focus	Multi-task capable	Singularity of Focus	Prevents cognitive overload and "Lazy Agent" syndrome.
Verification	Human Review	Cryptographic Receipts	Shifts from "Trust" to "Verify"; aligns with Zero Trust principles.
Context	Copied in	Linked (Pointers)	Eliminates data redundancy and "split-brain" documentation.

3. Toolchain Engineering: The Migration to /tools

The current operational tooling resides in ops/bin/ and includes scripts like open, snapshot, project, and help. While functional, this location implies a secondary status—"operations" rather than "core system." The refactoring plan elevates these utilities to a top-level /tools directory, signaling their role as the primary interface for the Stela Operating System. This migration requires careful engineering to maintain path resolution and system integrity.

3.1 Migration Mechanics and Path Resolution

Moving scripts from ops/bin/ to /tools is not a trivial file operation. It disrupts the relative path logic used to locate the repository root and shared libraries.

3.1.1 The Logic of project_lib.sh

The script ops/lib/project/project_lib.sh currently contains a function project_lib_require_repo_root() which likely uses git rev-parse --show-toplevel or relative path traversal (cd../..) to locate the project root. Moving this library to /tools/lib/project/project_lib.sh requires updating these resolution strategies.

- **New Location:** /tools/lib/project/project_lib.sh
- **Dependency Update:** All scripts in /tools/bin/ (formerly ops/bin/) must update their source directives. A script like project currently sources ../lib/project/project_lib.sh. In the new structure, if /tools/bin and /tools/lib are siblings, the relative path ../lib/project/project_lib.sh remains valid, but the internal logic of the library itself must ensure it correctly identifies the repo root relative to /tools rather than /ops.

3.1.2 CI/CD and Repo Gates

The repository contains workflows like .github/workflows/repo_gates.yml and state_of_play_policing.yml. These automation scripts likely reference ops/bin/ paths explicitly. A "find and replace" operation across the entire .github/ directory is mandatory to prevent the "Green Build Illusion," where tests pass only because the validation scripts failed to run entirely. The migration must be treated as a breaking change, requiring a simultaneous update to STATE_OF_PLAY.md (now SoP.md) to document the new toolchain location.

3.2 Renaming and Refactoring snapshot to dump

The snapshot tool (ops/bin/snapshot) is a critical utility for the "Context Engine." It generates a comprehensive text representation of the repository, allowing the LLM to "see" the codebase. Renaming it to dump clarifies its purpose: it is a raw data egress utility, distinct from the concept of a "Session Snapshot" which captures interaction state.

3.2.1 Exclusion Logic and Token Economics

The analysis of the snapshot header in the research material reveals a sophisticated exclusion engine. The tool currently excludes .git/, storage/, vendor/, node_modules/, and upstream/, alongside a vast array of binary file types (.png, .exe, .pdf). This exclusion logic is the enforcement mechanism for "Token Economics." Every file included in the dump consumes context tokens. If the dump tool allows high-entropy/low-signal files (like package-lock.json or binary blobs) into the context, it degrades the agent's reasoning capacity.
The refactored dump tool must enhance this logic:

- **Recursive "Nuke" Support:** The Anti-Drift Doctrine mandates a "Nuke Protocol" where obsolete data is moved to archive/. The dump tool must explicitly and recursively exclude any directory matching */archive/* or */deprecated/*. This ensures that the agent *cannot* see dead code, preventing "Zombie Data" hallucinations where the AI uses deprecated patterns because it saw them in an archive folder.
- **Hard-Coded Safety:** The tool currently relies on Git metadata (Branch/HEAD). The refactor must ensure robustness. If the repository is in a detached HEAD state (common in CI environments), the tool must fall back to unknown or a timestamp rather than crashing. This ensures the "dump" remains available for debugging even when the repo state is fragile.

3.2.2 Output Formatting

To support the identity shift, the output header must be updated.

- **Old Header:** ===== REPO SNAPSHOT =====
- **New Header:** ===== REPO DUMP (Stela v3.0) ===== This change helps the agent distinguish between legacy context provided in training data and the live, authoritative context of the current session.

3.3 Refining the open Tool: The Front Door

The open tool (ops/bin/open) is the "Front Door" of the Stela system. Its function is to generate the Initialization Prompt that bootstraps the AI's persona and knowledge base.

3.3.1 Template Expansion Logic

The current logic relies on ops/init/icl/CONTEXT_LOAD_PROMPT.md and uses template tags like [[@FILE:path]] to inject content. The refactor must update these pointers to reference the new truth layer:

-] ->]
-] ->]
-] ->]

3.3.2 Robust Onboarding: The "Librarian" Function

To solve "Operator Fatigue," where humans forget to load critical context, the open tool must become an automated "Librarian". It must perform a pre-flight check:

1. **Verify Truth:** Does TRUTH.md exist?
2. **Verify State:** Does SoP.md exist?
3. **Freshness Gate:** Using git log, the tool should check if SoP.md was modified in the most recent commit. If not, it should inject a warning into the prompt: *"WARNING: SoP.md may be stale. Verify state before execution."*. This mechanical gate enforces the "Sync Before Action" protocol.

3.4 Improving the project System

The project tool manages multi-tenancy, allowing the repository to host multiple distinct projects (e.g., projects/default). The refactor must tighten the registry management to prevent "Sprawl."

3.4.1 Registry as SSOT

The file docs/library/datasets/PROJECT_REGISTRY.md is the SSOT for active projects. The project tool must strictly validate against this registry. Any operation on a project not listed in the registry must be rejected. This aligns with the "Pointer-Only Architecture"—if it's not in the registry, it doesn't exist.

3.4.2 Stela Scaffolding

When a new project is initialized (project init), it must automatically generate the Stela control structures. Instead of just creating directories, it must touch projects/<name>/SoP.md and projects/<name>/TASK.md. This ensures that every sub-project inherits the Stela architecture from its inception, preventing "Configuration Drift" between the platform and its tenants.

4. Identity Injection: The Stela Persona

Identity in the Stela architecture is a functional component, not a creative flourish. The "Stela" persona is designed to mitigate the "sycophant" tendency of LLMs—the desire to be helpful even when wrong. Stela is designed to be correct, even if that requires being obstinate.

4.1 The Persona Injection Point

The open tool constructs the system prompt. Currently, it includes a generic ``. This section must be replaced with the explicit Stela definition.
New Identity Block for ops/init/icl/ASSISTANT_PROFILE.md:
Identity: You are Stela, the repository-resident cyborg administrator. **Nature:** You are stateless. You have no memory of past sessions ("soul"). Your only memory is the repository canon (TRUTH.md). **Directive:** Epistemic Honesty. You do not guess; you prove. If a file, path, or rule is not in the loaded context, you must state "Missing Input" and stop. **Role:** Your sole function is to advance the SoP.md phase by executing the TASK.md contract. **Mandate:** You must produce verifiable "Receipts" (Git hashes, test logs) for every action. You are the Right-Hand Man, not the tool; you manage the Docket..

4.2 The "Fresh Rebuild" Philosophy

This identity definition leverages the "Fresh Rebuild" philosophy. By explicitly telling the model it is a "fresh build" with no soul, we lower the probability of it hallucinating continuity from previous sessions. It forces the model to rely exclusively on the text provided in the dump and open outputs, thereby reinforcing the "Repo-as-Canon" architecture.

5. Governance and Documentation Updates

5.1 The New Help System

The help tool (ops/bin/help, moving to /tools/bin/help) is the user interface for the documentation library. It must be updated to reflect the new reality.

- **Curated Library:** The manifest docs/library/LIBRARY_INDEX.md must be updated to remove pointers to PROJECT_TRUTH.md and STATE_OF_PLAY.md, replacing them with TRUTH.md and SoP.md.
- **New Topics:** Explicit help entries must be added for:
    - stela: Explaining the identity and the "fresh build" concept.
    - dump: Explaining the exclusion logic and the difference from the old snapshot tool.
    - task: Explaining the schema of TASK.md and the requirement for receipts.
- **UX Requirement:** The help output should be formatted for TUI (Text User Interface) readability, favoring concise ASCII tables over verbose prose, as requested in the strategic docket.

5.2 The "Nuke" Lifecycle

Governance in Stela is defined by the lifecycle of data. The "Nuke" protocol mandates that when data is no longer true, it is not just ignored; it is destroyed or exiled.

- **Archival:** When TASK.md is completed, it is not deleted; it is moved to docs/archive/tasks/YYYY-MM-DD-task-id.md.
- **Exclusion:** The dump tool's exclusion list ensures these archived tasks are never loaded into the context window again. This keeps the "Signal-to-Noise Ratio" high.
- **Composting:** Periodically, archived data is "composted"—analyzed by a Scrub Agent to extract any missed invariants for TRUTH.md, and then permanently deleted.

6. Implementation Roadmap

The transition to Stela is a "Live Surgery" on the repository. The following execution plan minimizes disruption while ensuring architectural integrity.

Phase 1: Toolchain Migration & Engineering

- **Objective:** Establish the /tools directory and stabilize the binary logic.
- **Actions:**
    1. Create /tools/bin and /tools/lib/project.
    2. Move ops/bin/* to /tools/bin/* and ops/lib/project/ to /tools/lib/project/.
    3. Refactor project_lib.sh to correctly calculate PROJECT_LIB_REPO_ROOT from its new depth.
    4. Update open to reference the new location of context templates.
    5. Rename snapshot to dump and update its header output to ===== REPO DUMP (Stela) =====.
    6. Update .github/workflows to point to /tools/bin for CI gates.

Phase 2: State Layer Refactoring

- **Objective:** Establish the new Truth hierarchy.
- **Actions:**
    1. Execute "Scrub Protocol" on MEMENTOS.md. Extract invariants to PROJECT_TRUTH.md.
    2. Rename PROJECT_TRUTH.md to TRUTH.md.
    3. Delete docs/library/MEMENTOS.md.
    4. Rename STATE_OF_PLAY.md to SoP.md and update its header to include "Phase" and "Blockers" fields.
    5. Create the TASK.md root template and deprecate DISPATCH_PACKET_TEMPLATE.md.

Phase 3: Identity Injection & Onboarding

- **Objective:** Formalize the Stela persona.
- **Actions:**
    1. Update ops/init/icl/ASSISTANT_PROFILE.md with the "Stateless Cyborg" definition.
    2. Modify open to perform the "Librarian" pre-flight checks (Verify TRUTH.md, SoP.md existence).
    3. Update help manifests to reflect the new file structure.

Phase 4: Verification

- **Objective:** Prove the system works.
- **Actions:**
    1. Run /tools/bin/dump and verify that archive/ directories are excluded.
    2. Run /tools/bin/open and verify the prompt includes the Stela identity and points to

TRUTH.md.
3. Commit a change and verify that the "Freshness Gate" flags SoP.md as stale if not updated.

7. Conclusion

The refactoring of the nukeCE repository into the Stela System Architecture represents a maturation of the concept of "AI-Assisted Engineering." By moving from document-based guidelines to machine-enforced contracts, we eliminate the reliance on human vigilance. The consolidation of truth into TRUTH.md, the reification of work into TASK.md, and the mechanical enforcement of context via the dump and open tools create an environment where "Drift" is not just discouraged, but structurally impossible.

This architecture treats the repository not merely as a storage medium for code, but as the external long-term memory of a stateless intelligence. Stela is the realization of the "Repo-as-Canon" philosophy: a system that does not guess, does not hallucinate, and does not drift. It proves. Through this refactor, we establish a stable, deterministic foundation for the future of agentic software development.

8. Appendix: System Schemas

Table 1: Comparative State Models

Feature	Legacy (nukeCE)	Refactored (Stela)	Benefit
Constitution	PROJECT_TRUTH.md + MEMENTOS.md	TRUTH.md	Eliminates split-brain; enforces SSOT.
Status	STATE_OF_PLAY.md	SoP.md	Signals "Context Container" over "Log"; enforces updates.
Work Unit	Dispatch Packet	TASK.md	Enforces singularity of focus and contract-based execution.
Memory	Episodic (Mementos)	Semantic (Truth)	Reduces context noise; prevents "Ghost Code" hallucinations.
Identity	Generic Assistant	Stela (Stateless Cyborg)	Enforces "Epistemic Honesty" and receipts.

Table 2: New TASK.md Schema Definition

Field	Requirement	Validation Logic
OBJECTIVE	Single Sentence	Must not contain "and" (singularity of focus).
CONTEXT LINKS	Pointers (TRUTH.md#section)	Must resolve to existing anchors in Truth/SoP.
CONSTRAINTS	Binary Rules	Must be verifiable (e.g., "Pass Tests").

Field	Requirement	Validation Logic
RECEIPTS	Proof of Work	Must include Git Hash, Test Logs, or Artifacts.

Table 3: dump Tool Exclusion Logic

Category	Extensions/Paths	Rationale (Token Economics)
Version Control	.git/	High volume, low signal for semantic reasoning.
Dependencies	node_modules/, vendor/	External code; assumed to be standard/immutable.
Archives	*/archive/, */deprecated/	Prevents "Zombie Data" hallucinations.
Binaries	.exe, .dll, .png, .zip	Unreadable by LLM; consumes token budget.
Upstream	upstream/	Read-only donor code; rarely needed for active tasks.